Artificial Intelligence
Lecture 8: First-Order Logic

Prof. Dr. Pierre-Alexandre Murena

SoSe 2025

Limitations of propositional logic

Propositional logic is very adapted to describe properties of individual entities.

It is difficult (or even impossible) to use when describing properties of a group of entities. For instance:

- "The Wumpus is in only one cell" requires n^4 propositions!
- "My friends' friends are also my friends" requires to describe the friendship relationship with each person one by one. In particular, this description does not extend to a new person.

Outline

1. Syntax of first-order logic
2. Semantics of first-order logic
3. Knowledge representation in first-order logic
4. Using first-order logic

Symbols

Symbols are the base elements used to build PO sentences. However, they are **not** syntactically correct!

Symbols are of several types:

- **Variable symbols:** $x, y, z, \ldots$
- n-ary **function symbols**: $f, g, \ldots$
- **Constant symbols:** a, b, c
- n-ary **predicate symbols:** $P, Q, \ldots$
- **Logical symbols:**
  - **Logical connectors:** $\neg, \vee, \wedge, \vee, \rightarrow, \leftrightarrow$
  - **Logical quantifiers:** $\forall, \exists$
  - **Equality:** $=$

Terms

Intuitively, a **term** designates an entity. It can be:

- a variable x;
- a constant: for instance `TUHH`
- a function application: for instance `president(TUHH)`

```
t ::= < variable > | < constant > | < function >(< term >, < term >, ...)
```

Terms

Intuitively, a **term** designates an entity. It can be:

- a variable x;
- a constant: for instance `TUHH`
- a function application: for instance `president(TUHH)`

```
t ::= < variable > | < constant > | < function >(< term >, < term >, ...)
```

Remark: A term cannot be attributed a truth value! It makes no sense to say that `TUHH` is true or false,

Atoms

An **atom** is the smallest formula that cannot be cut. Atoms are predicates applied to terms:

```
< atom > ::= < predicate >(< term >, < term >, ...)
```

Example: `is_University(TUHH)`, `directed(George_Miller, Mad_Max_Furiosa)` etc

Remark: Atoms being formalae, they are associated to truth values.

Formulae

A **formula** (or **sentence**) can be:

- **An atom**: $\texttt{directed(George_Miller, Mad_Max_Furiosa)}$
- **An identity formula**: $x = \texttt{George_Miller}$
- **Propositional connective**: $\neg\, \texttt{directed(Quentin_Tarantino, Mad_Max_Furiosa)}$, $\texttt{directed(Quentin_Tarantino, Mad_Max_Furiosa)} \lor \texttt{directed(George_Miller, Mad_Max_Furiosa)}$
- **Quantification:**
  $\forall x\ \texttt{directed(George_Miller, } x\texttt{)} \rightarrow \neg\, \texttt{directed(Quentin_Tarantino, } x\texttt{)}$

Formulae

```
< formula > ::=  < atom >
                 | < term > = < term >
                 | ¬ < formula >
                 | < formula > < binary_connector > < formula >
                 | < quantifier > < variable > < formula >
```

Free and bound variables

Consider the following formula:

$$\forall x \; P(x, y)$$

We can see that the variables x and y play a different role:

- Variable x is within the reach of quantifier $\forall$. We say it is **bound**.
- Variable y is not bound by a quantifier. We say it is **free**.

Free and bound variables

The notion of **free variable** can be defined more rigorously:

$$
\begin{aligned}
free(x) &= \{x\} \\
free(f(t_1, \ldots, t_n) &= free(t_1) \cup \ldots \cup free(t_n) \\
free(t_1 = t_2) &= free(t_1) \cup free(t_2) \\
free(P(t_1, \ldots, t_n)) &= free(t_1) \cup \ldots \cup free(t_n) \\
free(\neg\phi) &= free(\phi) \\
free(\phi * \psi) &= free(\phi) \cup free(\psi) \quad \text{for } * = \vee, \wedge, \rightarrow, \leftrightarrow \\
free(Qx\phi) &= free(\phi) \setminus \{x\} \quad \text{for } Q = \forall, \exists
\end{aligned}
$$

Remark: Formulae without free variables are called **closed formulae**. Formulae containing free variables are called **open formulae**.

Outline

1. Syntax of first-order logic
2. Semantics of first-order logic
3. Knowledge representation in first-order logic
4. Using first-order logic

Reminder: Semantics in propositional logic

In **Propositional logic**, the semantics of a proposition is defined in two steps:

- **Step 1:** Attributing a truth value (*interpretation*) to each propositional variable
- **Step 2:** Defining the truth value of compound sentences

This was possible because we were working at the level of objects directly.

Conversely, FOL considers functions and relations over objects.

Interpretations

An interpretation $\mathcal{I}$ is a tuple $\mathcal{I} = \langle \mathcal{D}, .^{\mathcal{I}} \rangle$, where:

- $\mathcal{D}$ is a non-empty set called **domain of interpretation**
- $.^{\mathcal{I}}$ is a function which maps:
  - n-ary function symbols f to n-ary functions $f^{\mathcal{I}} \in [\mathcal{D}^n \to \mathcal{D}]$
  - Constant symbols a to objects $a^{\mathcal{I}} \in \mathcal{D}$
  - n-ary predicates P to n-ary relations $P^{\mathcal{I}} \subseteq \mathcal{D}^n$.

Interpretations

An interpretation $\mathcal{I}$ is a tuple $\mathcal{I} = \langle \mathcal{D}, .^{\mathcal{I}} \rangle$, where:

- $\mathcal{D}$ is a non-empty set called **domain of interpretation**
- $.^{\mathcal{I}}$ is a function which maps:
  - n-ary function symbols f to n-ary functions $f^{\mathcal{I}} \in [\mathcal{D}^n \to \mathcal{D}]$
  - Constant symbols a to objects $a^{\mathcal{I}} \in \mathcal{D}$
  - n-ary predicates P to n-ary relations $P^{\mathcal{I}} \subseteq \mathcal{D}^n$.

Interpretation of terms:

$$(f(t_1, \ldots, t_n))^{\mathcal{I}} = f^{\mathcal{I}}(t_1^{\mathcal{I}}, \ldots, t_n^{\mathcal{I}})$$

Truth of ground atoms: (with no variable)

$$\mathcal{I} \models P(t_1, \ldots, t_n) \text{ iff } \langle t_1^{\mathcal{I}}, \ldots, t_n^{\mathcal{I}} \rangle \in P^{\mathcal{I}}$$

Example

We define the interpretation $\mathcal{I}$ as follows:

- **Domain:** $\mathcal{D} = \{o_1, \ldots, o_n\}$
- **Constant:** $\texttt{George_Miller}^{\mathcal{I}} = o_1$
- **Constant:** $\texttt{Mad_Max_Furiosa}^{\mathcal{I}} = o_2$
- **Constant:** $\texttt{Dune}^{\mathcal{I}} = o_3$
- **Predicate:** $\texttt{Directed}^{\mathcal{I}} = \{(o_1, o_2)\}$

We have:

$$\mathcal{I} \models \texttt{Directed(George_Miller, Mad_Max_Furiosa)}$$

$$\mathcal{I} \not\models \texttt{Directed(George_Miller, Dune)}$$

Variable assignment

Let $\mathcal{V}$ be the set of variables.

A **variable assignment** is a function $\alpha : \mathcal{V} \to \mathcal{D}$.

Interpretation of terms under $\mathcal{I}, \alpha$:

$$x^{\mathcal{I},\alpha} = \alpha(x)$$
$$a^{\mathcal{I},\alpha} = a^{\mathcal{I}}$$
$$(f(t_1, \ldots, t_n)) = f^{\mathcal{I}}(t_1^{\mathcal{I},\alpha}, \ldots, t_1^{\mathcal{I},\alpha})$$

Truth of atomic formulae: (with variables and constants)

$$\mathcal{I}, \alpha \models P(t_1, \ldots, t_n) \text{ iff } \langle t_1^{\mathcal{I},\alpha}, \ldots, t_n^{\mathcal{I},\alpha} \rangle \in P^{\mathcal{I}}$$

Example

We define the interpretation $\mathcal{I}$ as follows:

- **Domain:** $\mathcal{D} = \{o_1, \ldots, o_n\}$
- **Constant:** $\texttt{George_Miller}^{\mathcal{I}} = o_1$
- **Constant:** $\texttt{Mad_Max_Furiosa}^{\mathcal{I}} = o_2$
- **Constant:** $\texttt{Dune}^{\mathcal{I}} = o_3$
- **Predicate:** $\texttt{Directed}^{\mathcal{I}} = \{(o_1, o_2)\}$

and **variable assignment** $\alpha = \{x \mapsto o_1\}$

We have:

$$\mathcal{I}, \alpha \models \texttt{Directed}(x,\ \texttt{Mad_Max_Furiosa})$$

Truth

The truth of formula ϕ under $\mathcal{I}$ and α is defined as follows:

$$
\begin{aligned}
\mathcal{I}, \alpha \models P(t_1, \ldots, t_n) \quad & \text{iff} \quad \langle t_1^{\mathcal{I},\alpha}, \ldots, t_n^{\mathcal{I},\alpha} \rangle \in P^{\mathcal{I}} \\
\mathcal{I}, \alpha \models t_1 = t_2 \quad & \text{iff} \quad t_1^{\mathcal{I},\alpha} = t_2^{\mathcal{I},\alpha} \\
\mathcal{I}, \alpha \models \neg\phi \quad & \text{iff} \quad \mathcal{I}, \alpha \not\models \phi \\
\mathcal{I}, \alpha \models \phi \wedge \psi \quad & \text{iff} \quad \mathcal{I}, \alpha \models \phi \text{ and } \mathcal{I}, \alpha \models \psi \\
\mathcal{I}, \alpha \models \phi \vee \psi \quad & \text{iff} \quad \mathcal{I}, \alpha \models \phi \text{ or } \mathcal{I}, \alpha \models \psi \\
\mathcal{I}, \alpha \models \phi \rightarrow \psi \quad & \text{iff} \quad \text{if } \mathcal{I}, \alpha \models \phi \text{ then } \mathcal{I}, \alpha \models \psi \\
\mathcal{I}, \alpha \models \phi \leftrightarrow \psi \quad & \text{iff} \quad \mathcal{I}, \alpha \models \phi \text{ iff } \mathcal{I}, \alpha \models \psi \\
\mathcal{I}, \alpha \models \forall x \phi \quad & \text{iff} \quad \mathcal{I}, \alpha[x \setminus d] \models \phi \text{ for all } d \in \mathcal{D} \\
\mathcal{I}, \alpha \models \exists x \phi \quad & \text{iff} \quad \mathcal{I}, \alpha[x \setminus d] \models \phi \text{ for some } d \in \mathcal{D}
\end{aligned}
$$

with $\alpha[x \setminus d](u) = \alpha(u)$ if $u \neq x$ and d otherwise.

Example

We define the interpretation $\mathcal{I}$ as follows:

- **Domain:** $\mathcal{D} = \{o_1, \ldots, o_n\}$
- **Constant:** $\texttt{George_Miller}^{\mathcal{I}} = o_1$
- **Constant:** $\texttt{Mad_Max_Furiosa}^{\mathcal{I}} = o_2$
- **Constant:** $\texttt{Dune}^{\mathcal{I}} = o_3$
- **Predicate:** $\texttt{Directed}^{\mathcal{I}} = \{(o_1, o_2)\}$

and **variable assignment** $\alpha = \{x \mapsto o_1, y \mapsto o_3\}$

We have:

$$\mathcal{I}, \alpha \models \texttt{Directed}(x,\ \texttt{Mad_Max_Furiosa}) \wedge \neg\texttt{Directed}(x, y)$$

Example

We define the interpretation $\mathcal{I}$ as follows:

- **Domain:** $\mathcal{D} = \{o_1, \ldots, o_n\}$
- **Constant:** $\texttt{George_Miller}^{\mathcal{I}} = o_1$
- **Constant:** $\texttt{Mad_Max_Furiosa}^{\mathcal{I}} = o_2$
- **Constant:** $\texttt{Dune}^{\mathcal{I}} = o_3$
- **Predicate:** $\texttt{Directed}^{\mathcal{I}} = \{(o_1, o_2)\}$

and **variable assignment** $\alpha = \{x \mapsto o_1, y \mapsto o_3\}$

We have:

$$\mathcal{I}, \alpha \models \exists u\ \texttt{Directed}(u,\ \texttt{Mad_Max_Furiosa})$$

Models, entailment and logical equivalence

$\mathcal{I}, \alpha$ is a **model** of ϕ
if and only if
$\mathcal{I}, \alpha \models \phi$.

Models, entailment and logical equivalence

$\mathcal{I}, \alpha$ is a **model** of ϕ
if and only if
$\mathcal{I}, \alpha \models \phi$.

Formula ϕ **entails** formula ψ (written $\phi \models \psi$)
if and only if
if $\mathcal{I}, \alpha \models \phi$, then if $\mathcal{I}, \alpha \models \psi$

Models, entailment and logical equivalence

$\mathcal{I}, \alpha$ is a **model** of ϕ
if and only if
$\mathcal{I}, \alpha \models \phi$.

Formula ϕ **entails** formula ψ (written $\phi \models \psi$)
if and only if
if $\mathcal{I}, \alpha \models \phi$, then if $\mathcal{I}, \alpha \models \psi$

Formulae ϕ and ψ are **logically equivalent** (written $\phi \equiv \psi$)
if and only if
if $\phi \models \psi$ and $\psi \models \phi$

Outline

1. Syntax of first-order logic
2. Semantics of first-order logic
3. Knowledge representation in first-order logic
4. Using first-order logic

From the real world to FOL

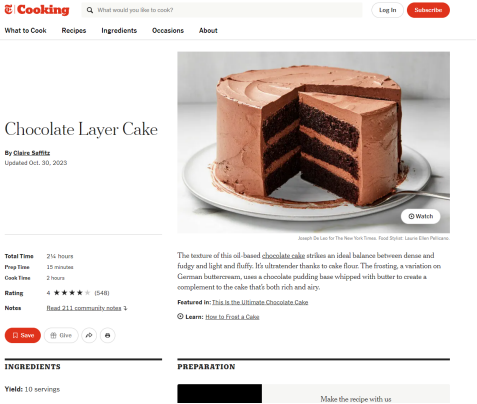
The formalism of first-order logic offers a powerful way of describing entities and relations in a domain, as well as to reason about them.

However, the domain is not naturally structured, so we need to make the design choices and to create the knowledge base. This task is called **knowledge engineering**.

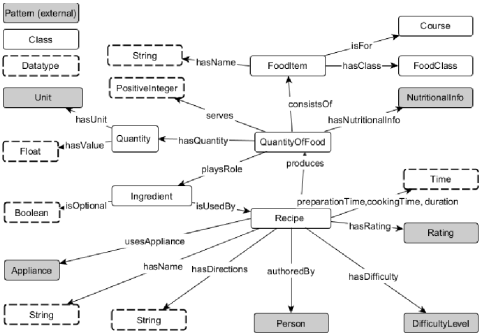
Plan

1. Syntax of first-order logic
2. Semantics of first-order logic
3. Knowledge representation in first-order logic
   - Knowledge engineering
   - Example: The Wumpus world
4. Using first-order logic

An example of knowledge engineering



→



The challenges of knowledge engineering

Knowledge engineering requires to carefully:

- **Identify the task:** knowing what is the kind of knowledge that will be needed, what are the pursued goals etc
- **Assemble the relevant knowledge:** Collecting observations about the domain, interacting with domain experts...
- **Decide on the vocabulary of predicates, functions and constants:** Translating the knowledge from natural language to FOL language
- **Encode general knowledge about the domain:** Writing down the axioms for the vocabulary terms. *Usually where the conception issues appear*
- **Encode a description of the specific problem instance, pose queries, get answers...** *and debut the knowledge base!*

Plan

1 Syntax of first-order logic

2 Semantics of first-order logic

3 Knowledge representation in first-order logic
- Knowledge engineering
- Example: The Wumpus world

4 Using first-order logic

Perception

A perception can be represented as a triple $[Stench, Breeze, Glitter]$.

We represent the perception of agent at time t with the relation:

$$Percept([Stench, Breeze, Glitter], t)$$

The raw percept implies certain facts about the current state:

$$\begin{aligned} \forall t, s, g, &\quad Percept([s, Breeze, g], t) \rightarrow Breeze(t) \\ \forall t, b, g, &\quad Percept([Stench, b, g], t) \rightarrow Stench(t) \\ \forall t, s, b, &\quad Percept([s, b, Glitter], t) \rightarrow Glitter(t) \end{aligned}$$

Representing the environment

We need to code the location of the agent, the pits, the gold and the Wumpus. For that, we have two solutions:

Solution 1: As in propositional logic, we can associate one variable to each position (for instance $P_{1,1}, P_{1,2}$ etc).
This however makes it difficult to represent the notion of adjacency: it has to be defined for each pair of positions.

Solution 2: We code a position by a list $[x, y]$.
We then represent the contents of the cells with $At(o, [x, y], t)$ where o can be the agent, the Wumpus, a pit or the gold. For instance $At(Agent, [1, 3], t)$.

Time independent locations

The Wumpus does not change its position over time. This can be expressed with the rule:

$$\forall t_1, t_2, x, y \; At(Wumpus, [x, y], t_1) \rightarrow At(Wumpus, [x, y], t_2)$$

We have the same rules for the pits and the gold:

$$\forall t_1, t_2, x, y \; At(Pit, [x, y], t_1) \rightarrow At(Pit, [x, y], t_2)$$

$$\forall t_1, t_2, x, y \; At(Gold, [x, y], t_1) \rightarrow At(Gold, [x, y], t_2)$$

Unicity of locations

The agent can be at one place at a time:

$$\forall s_1, s_2, t \; At(Agent, s_1, t) \wedge At(Agent, s_2, t) \rightarrow s_1 = s_2$$

The Wumpus can be at one place at a time:

$$\forall s_1, s_2, t \; At(Wumpus, s_1, t) \wedge At(Wumpus, s_2, t) \rightarrow s_1 = s_2$$

Properties of cells

A cell can be breezy ($Breezy([x, y])$) or stenchy ($Stenchy([x, y])$). Unlike for the relation *Breeze*, *Breezy* is a property of a cell and not of the perception at time t. Both are related though:

$$\forall s, t \; At(Agent, s, t) \wedge Breeze(t) \rightarrow Breezy(s)$$

$$\forall s, t \; At(Agent, s, t) \wedge \neg Breeze(t) \rightarrow \neg Breezy(s)$$

$$\forall s, t \; At(Agent, s, t) \wedge Stench(t) \rightarrow Stenchy(s)$$

$$\forall s, t \; At(Agent, s, t) \wedge \neg Stench(t) \rightarrow \neg Stenchy(s)$$

Adjacent cells

An important relation in the Wumpus environment is the notion of adjacency of two cells:

$$\forall x, y, a, b\ Adjacent([x, y], [a, b]) \leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$

Adjacent cells

An important relation in the Wumpus environment is the notion of adjacency of two cells:

$$\forall x, y, a, b\ Adjacent([x, y], [a, b]) \leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$

The conditions on the breezy cells can now be expressed with one single axiom:

$$\forall s, t\ Breezy(s) \leftrightarrow \exists r\ Adjacent(r, s) \wedge At(Pit, r, t)$$

$$\forall s, t\ Stenchy(s) \leftrightarrow \exists r\ Adjacent(r, s) \wedge At(Wumpus, r, t)$$

Outline

1. Syntax of first-order logic
2. Semantics of first-order logic
3. Knowledge representation in first-order logic
4. Using first-order logic

Interaction with FOL agents

An agent based on first-order logic has a **knowledge base** which is filled with a sentences in FOL.

Sentences are added to the KB using TELL. Such sentences are called **assertions**:

$$\text{TELL}(KB, Director(George_Miller))$$

$$\text{TELL}(KB, Person(Quentin_Tarantino))$$

$$\text{TELL}(KB, \forall x \; Director(x) \rightarrow Person(x))$$

Interactions with FOL agents

Questions can be asked to the agent about its knowledge: such questions are called **queries**.

$$\textsc{Ask}(Person(George_Miller))$$

should reply true. *However, this requires some inference capabilities from the agent...*

$$\textsc{Ask}(\exists x\ Person(x))$$

should reply true, but is it satisfactory?

Interactions with FOL agents

Questions can be asked to the agent about its knowledge: such questions are called **queries**.

$$\textsc{Ask}(Person(George_Miller))$$

should reply true. *However, this requires some inference capabilities from the agent...*

$$\textsc{Ask}(\exists x\ Person(x))$$

should reply true, but is it satisfactory? Not really... We expect to know the values of x that makes it true:

$$\textsc{AskVars}(\exists x\ Person(x))$$

should reply $[x/George_Miller]$ and $[x/Quentin_Taranttino]$

Logic programming

Logic programming is a technology that aims to embody the declarative vision of logic:

- Systems are constructed by expressing knowledge in a formal language
- Problems are solved by running inference processes on that knowledge

Example: Last week, we saw how to express a search problem by only specifying a description of the problem and the goals in propositional logic.

Prolog

See the demonstration...

How does Prolog work?

Prolog solver is based on an inference algorithm called **backward induction**.

The principle of backward induction is to explore knowledge starting from the goal and retrieving the knowledge from the knowledge base.

Prolog clauses are **Horn clauses**, i.e. disjunctions of literals, at most one of which is positive. Horn clauses can be written as:

$$A_1 \wedge \ldots \wedge A_n \rightarrow B$$

To prove B, we then have to prove all $A_1, \ldots, A_n$.

Backward chaining: Example

Consider the following knowledge base:

$$father(x, y) \rightarrow parent(x, y)$$
$$mother(x, y) \rightarrow parent(x, y)$$
$$parent(x, z) \wedge parent(z, y) \rightarrow grandparent(x, z)$$
$$father(bob, alice)$$
$$mother(eve, alice)$$
$$father(mike, bob)$$
$$father(mike, ryan)$$
$$father(ryan, anna)$$

Question 1: $\textsc{AskVar}(\exists x\ mother(eve, x))$?

Backward chaining: Example

Consider the following knowledge base:

$$father(x, y) \rightarrow parent(x, y)$$
$$mother(x, y) \rightarrow parent(x, y)$$
$$parent(x, z) \wedge parent(z, y) \rightarrow grandparent(x, z)$$
$$father(bob, alice)$$
$$mother(eve, alice)$$
$$father(mike, bob)$$
$$father(mike, ryan)$$
$$father(ryan, anna)$$

Question 2: $\textsc{AskVar}(\exists x\ parent(eve, x))$?

Backward chaining: Example

Consider the following knowledge base:

$$father(x, y) \rightarrow parent(x, y)$$
$$mother(x, y) \rightarrow parent(x, y)$$
$$parent(x, z) \wedge parent(z, y) \rightarrow grandparent(x, z)$$
$$father(bob, alice)$$
$$mother(eve, alice)$$
$$father(mike, bob)$$
$$father(mike, ryan)$$
$$father(ryan, anna)$$

Question 3: $\textsc{AskVar}(\exists x\ grandparent(mike, x))$?